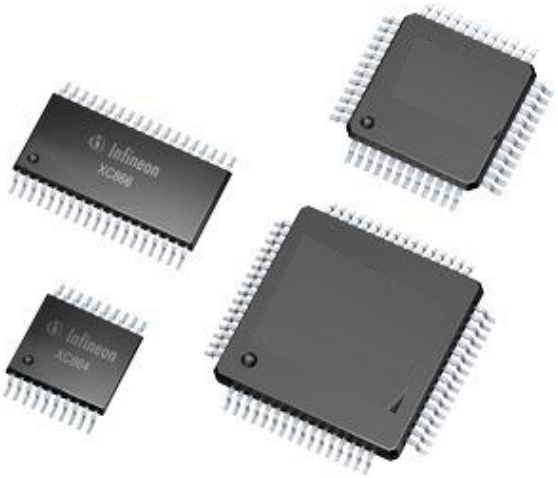
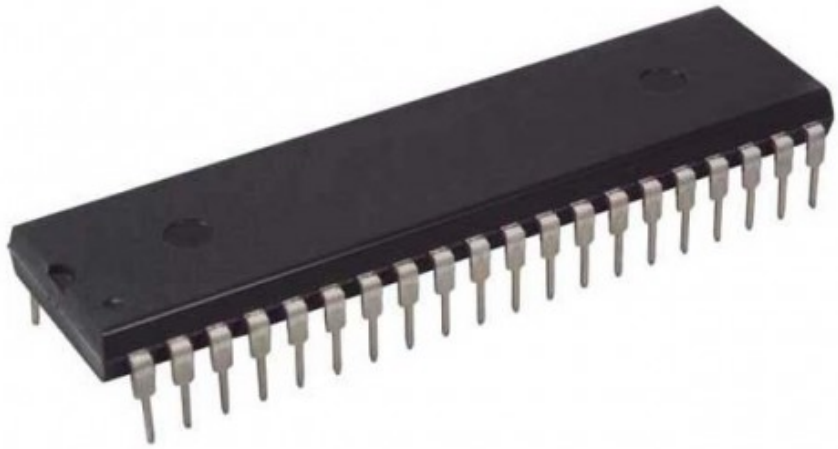
8051 Microcontroller





Interrupts in 8051

•An interrupt is an external or internal event that interrupts the microcontroller to inform it that a device needs its service.

A single microcontroller can serve several devices by two ways

Interrupts

•Whenever any device needs its service, the device notifies the microcontroller by sending it an interrupt signal

•Upon receiving an interrupt signal, the microcontroller interrupts whatever it is doing and serves the device

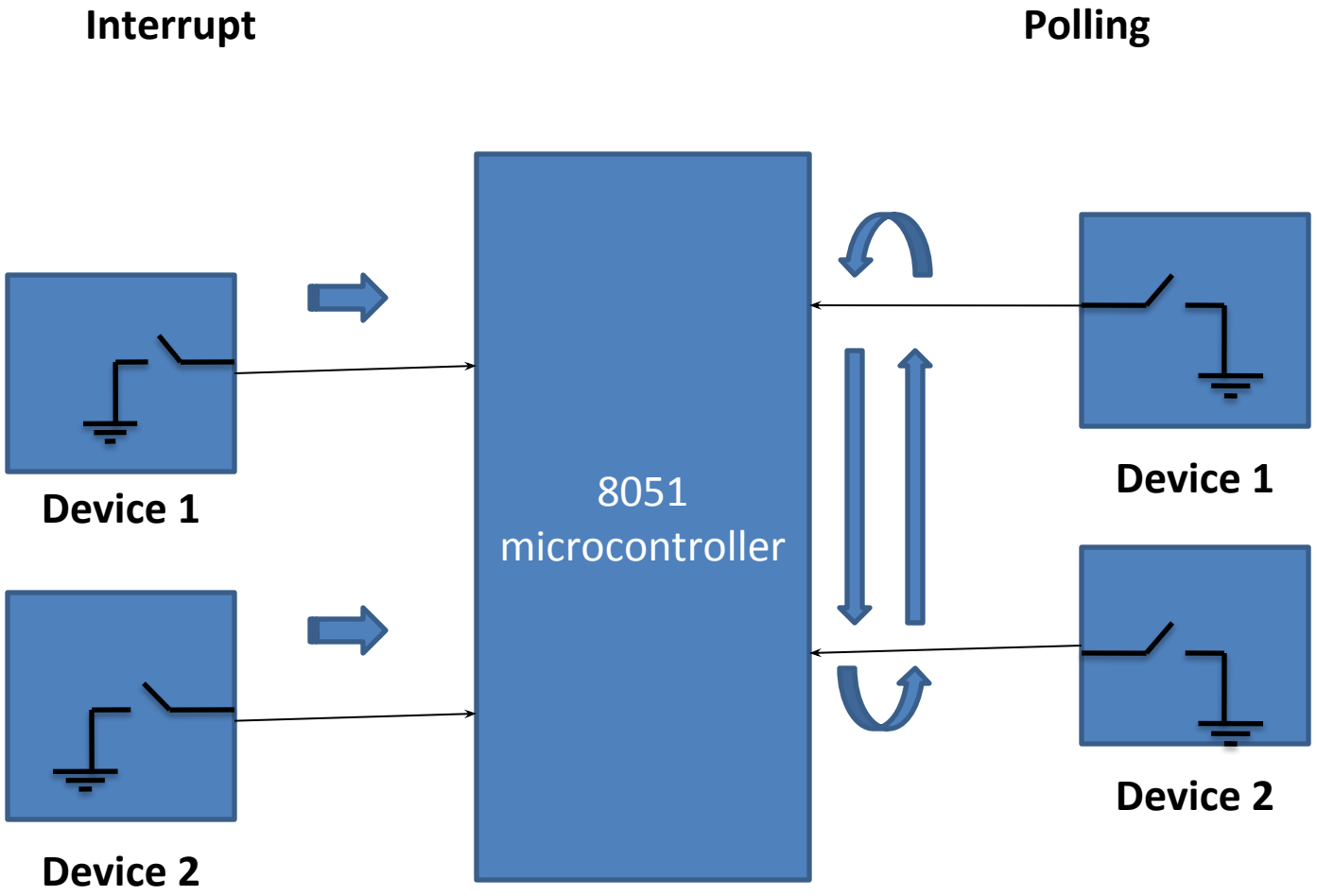
•The program which is associated with the interrupt is called the interrupt service routine (ISR) or interrupt handler

Polling

•The microcontroller continuously monitors the status of a given device

•When the conditions met, it performs the service

•After that, it moves on to monitor the next device until every one is serviced



Switch pressed: LED on
Switch released: LED off

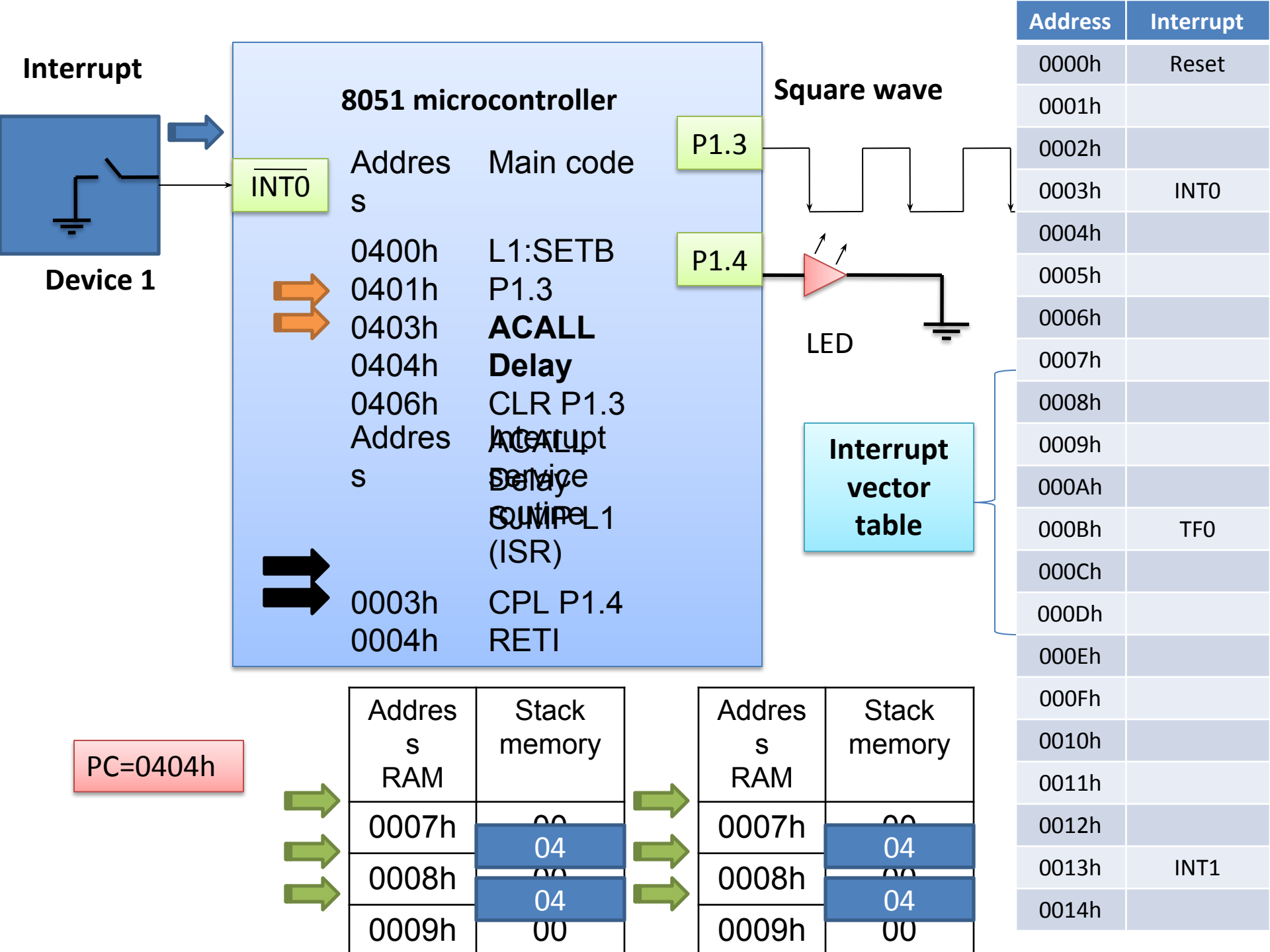
Table 11-1: Interrupt Vector Table for the 8051

Interrupt	ROM Location (Hex)	Pin	Flag Clearing
Reset	0000	9	Auto
External hardware interrupt 0 (INT0)	0003	P3.2 (12)	Auto
Timer 0 interrupt (TF0)	000B		Auto
External hardware interrupt 1 (INT1)	0013	P3.3 (13)	Auto
Timer 1 interrupt (TF1)	001B		Auto
Serial COM interrupt (RI and TI)	0023		Programmer clears it

- ROM memory locations 00 to 30H locations are for Interrupt vector table
- When an interrupt is invoked, the microcontroller runs the interrupt service Routine
- For every interrupt, there is a fixed location in memory that holds the address of its ISR
- The group of memory locations set aside to hold the addresses of ISRs is called interrupt vector table

Best way to write a code is:

```
ORG 00H
        LJMP MAIN          ;by-pass int. vector table
ORG 30H
        MAIN:
        …….
        …….
END
```



Address	Interrupt
0000h	Reset
0001h	
0002h	
0003h	INT0
0004h	
0005h	
0006h	
0007h	
0008h	
0009h	
000Ah	
000Bh	TF0
000Ch	
000Dh	
000Eh	
000Fh	
0010h	
0011h	
0012h	
0013h	INT1
0014h	

Steps in executing an interrupt

- It finishes the instruction it is executing and saves the address of next instruction (PC) on stack

- it jumps to fixed location in memory called interrupt vector table that holds the address of interrupt service routine.

- Microcontroller gets the address of ISR from interrupt vector table and jumps to it. It start to execute ISR until it reaches the last instruction of subroutine, which is RETI.

- Upon execution of RETI instruction, the microcontroller returns to place where it was interrupted. First it gets program counter (PC) address from stack by popping the top two bytes of stack in to PC. Then it starts to execute from address.

D7							D0
--	--	PT2	PS	PT1	PX1	PT0	PX0

Priority bit = 1 assigns high priority. Priority bit = 0 assigns low priority.

--	IP.7	Reserved
--	IP.6	Reserved
PT2	IP.5	Timer 2 interrupt priority bit (8052 only)
PS	IP.4	Serial port interrupt priority bit
PT1	IP.3	Timer 1 interrupt priority bit
PX1	IP.2	External interrupt 1 priority bit
PT0	IP.1	Timer 0 interrupt priority bit
PX0	IP.0	External interrupt 0 priority bit

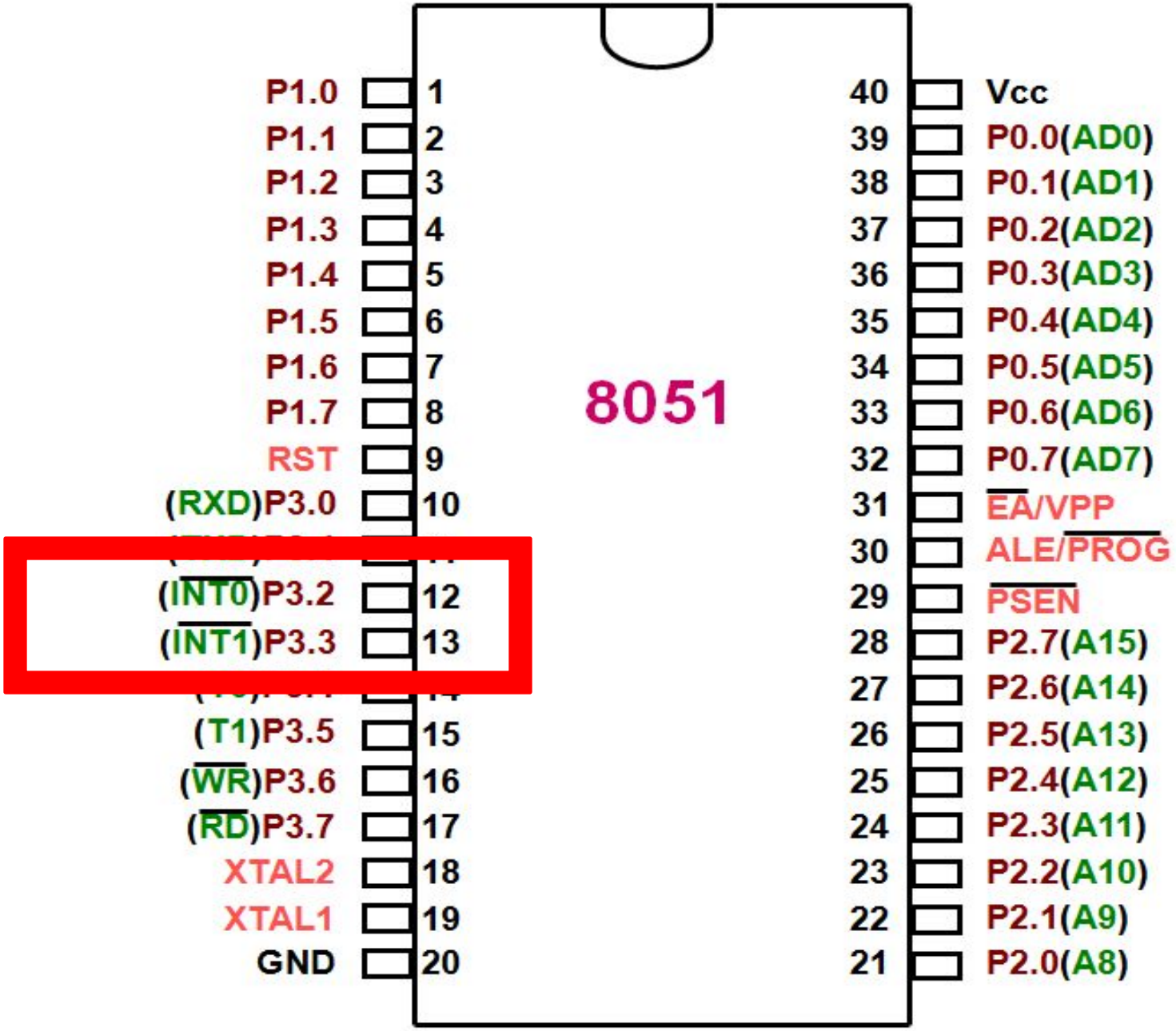
User software should never write 1s to unimplemented bits, since they may be used in future products.

IE (Interrupt enable) register:

D7							D0
EA	--	ET2	ES	ET1	EX1	ET0	EX0

EA	IE.7	Disables all interrupts. If EA = 0, no interrupt is acknowledged. If EA = 1, each interrupt source is individually enabled or disabled by setting or clearing its enable bit.
--	IE.6	Not implemented, reserved for future use.*
ET2	IE.5	Enables or disables Timer 2 overflow or capture interrupt (8052 only).
ES	IE.4	Enables or disables the serial port interrupt.
ET1	IE.3	Enables or disables Timer 1 overflow interrupt.
EX1	IE.2	Enables or disables external interrupt 1.
ET0	IE.1	Enables or disables Timer 0 overflow interrupt.
EX0	IE.0	Enables or disables external interrupt 0.

*User software should not write 1s to reserved bits. These bits may be used in future flash microcontrollers to invoke new features.

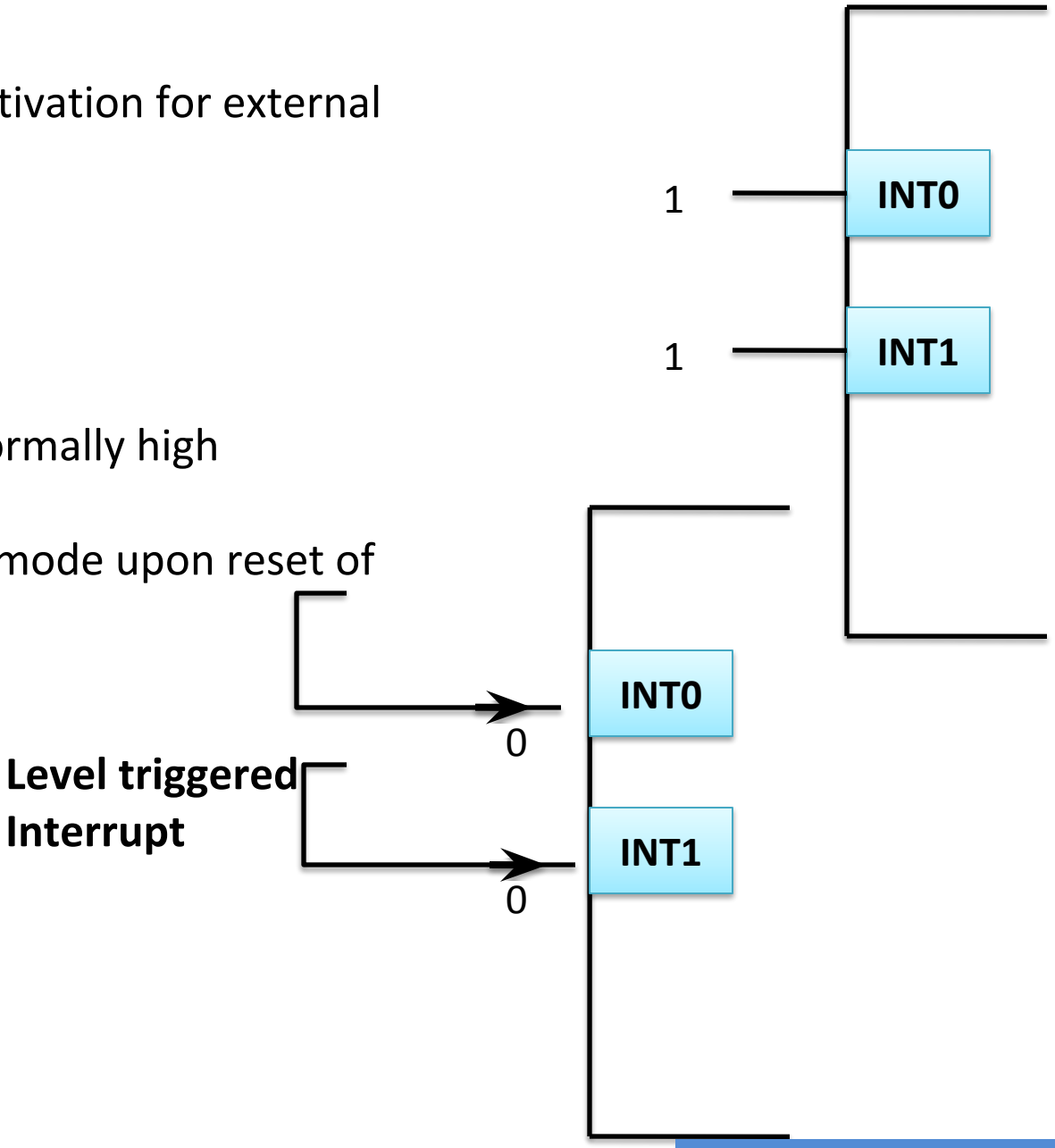

8051
P1.0 1
P1.1 2
P1.2 3
P1.3 4
P1.4 5
P1.5 6
P1.6 7
P1.7 8
RST 9
(RXD)P3.0 10
(INT0)P3.2 12
(INT1)P3.3 13
(T1)P3.5 15
(WR)P3.6 16
(RD)P3.7 17
XTAL2 18
XTAL1 19
GND 20
40 Vcc
39 P0.0(AD0)
38 P0.1(AD1)
37 P0.2(AD2)
36 P0.3(AD3)
35 P0.4(AD4)
34 P0.5(AD5)
33 P0.6(AD6)
32 P0.7(AD7)
31 EA/VPP
30 ALE/PROG
29 PSEN
28 P2.7(A15)
27 P2.6(A14)
26 P2.5(A13)
25 P2.4(A12)
24 P2.3(A11)
23 P2.2(A10)
22 P2.1(A9)
21 P2.0(A8)

There are two types of activation for external hardware interrupt

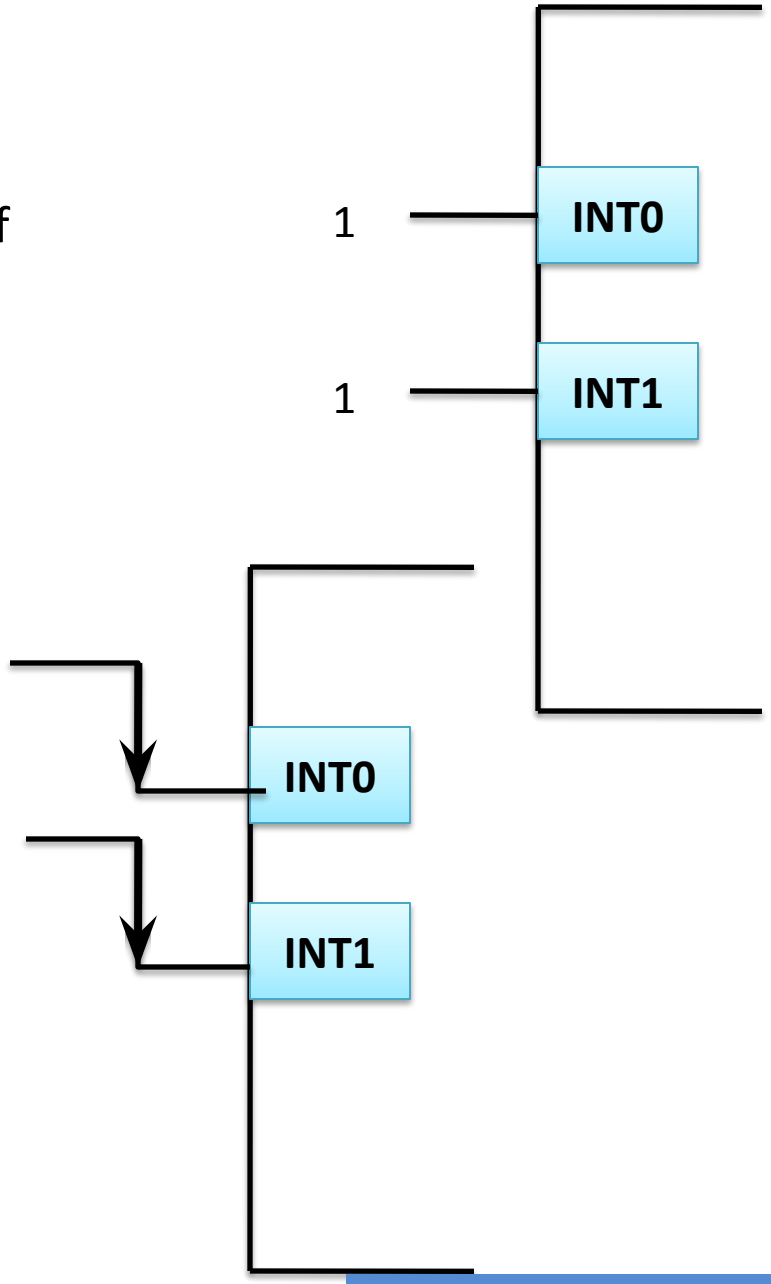
1.Level triggered
2.Edge triggered

- INT0 and INT1 pins are normally high

- Level triggered is default mode upon reset of 8051



- INT0 and INT1 pins are normally high
- Level triggered is default mode upon reset of 8051,
- to make them edge triggered interrupts, we must programs bits of TCON register



TCON



IT0 = 0, INT0 level triggered interrupt
IT0 = 1, INT0 edge triggered interrupt

IT1 = 0, INT1 level triggered interrupt
IT1 = 1, INT1 edge triggered interrupt

IE0 and IE1 are use to keep track of edge triggered interrupt, are used internally by 8051 to indicate whether or not an interrupt is in use.

Upon edge triggered pulse on INT0 and INT1, 8051 set IEx bit in TCON reg. Only execution of RETI instruction at the end of ISR will clear IEx

8051/52 Interrupt Priority Upon Reset

Highest to Lowest Priority	
External Interrupt 0	(INT0)
Timer Interrupt 0	(TF0)
External Interrupt 1	(INT1)
Timer Interrupt 1	(TF1)
Serial Communication	(RI + TI)
Timer 2 (8052 only)	**TF2**

LEVEL TRIGGERED INTERRUPT EXAMPLE

```
ORG 00H
        LJMP MAIN
ORG 13H
        SETB P3.1
        ACALL Delay
        CLR P3.1
        ACALL Delay
        RETI

ORG 30H
MAIN:   MOV IE,#84H    ; ENABLE EINT1
AGAIN:  SETB P1.1
        ACALL Delay
        CLR P1.1
        ACALL Delay
        SJMP AGAIN

END
```

EDGE TRIGGERED INTERRUPT EXAMPLE

```
ORG 0000H
        LJMP MAIN
ORG 13H
        CPL P3.1
        RETI

ORG 30H
MAIN:   MOV IE,#84H     ; ENABLE EINT1
        SETB IT1        ; EDGE TRIGGERED INTERRUPT
AGAIN:  SETB P1.1
        ACALL Delay
        CLR P1.1
        ACALL Delay
        SJMP AGAIN

END
```